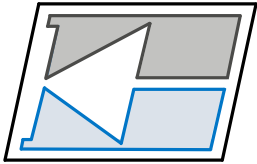


INSTITUTO FEDERAL
Santa Catarina



Departamento Acadêmico de Eletrônica – **DAELN**
IFSC Câmpus Florianópolis

Programação C++

Ponteiros

Prof. Matheus Leitzke Pinto
matheus.pinto@ifsc.edu.br

Sumário de aula

- Variáveis e ponteiros
- Aritmética de ponteiros
- Vetores e ponteiros
- O ponteiro do tipo "void *"
- Pontoeiro para funções

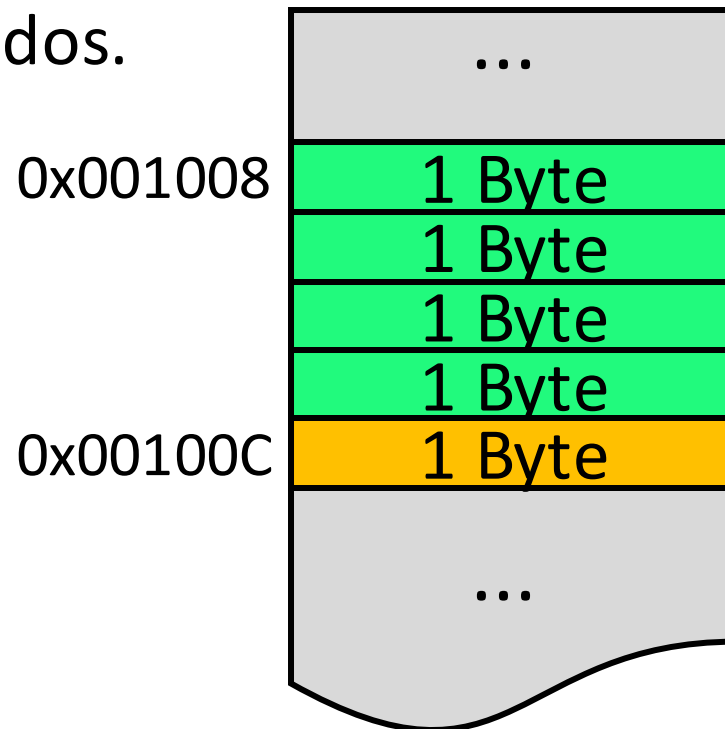


Ponteiros

Variáveis e ponteiros

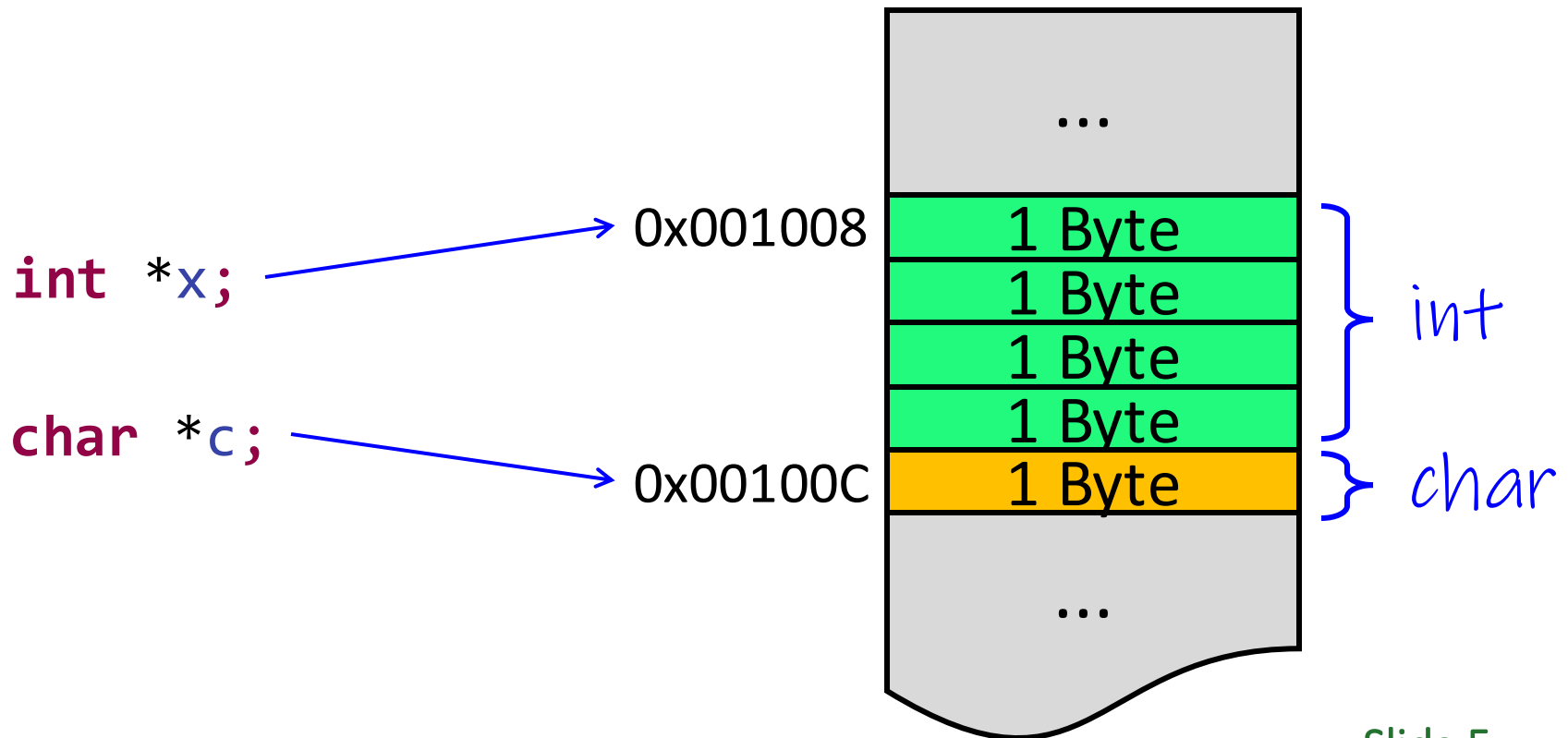
Variáveis e ponteiros

- Um **ponteiro** é uma variável que contém um valor que representa um endereço de memória.
- O tipo mais básico de ponteiro é o que contém o endereço de memória de um local de armazenamento de dados.



Variáveis e ponteiros

- Para declarar um ponteiro, devemos indicar o tipo de dado para que ele aponta, juntamente com o declarador de tipo asterisco (*).



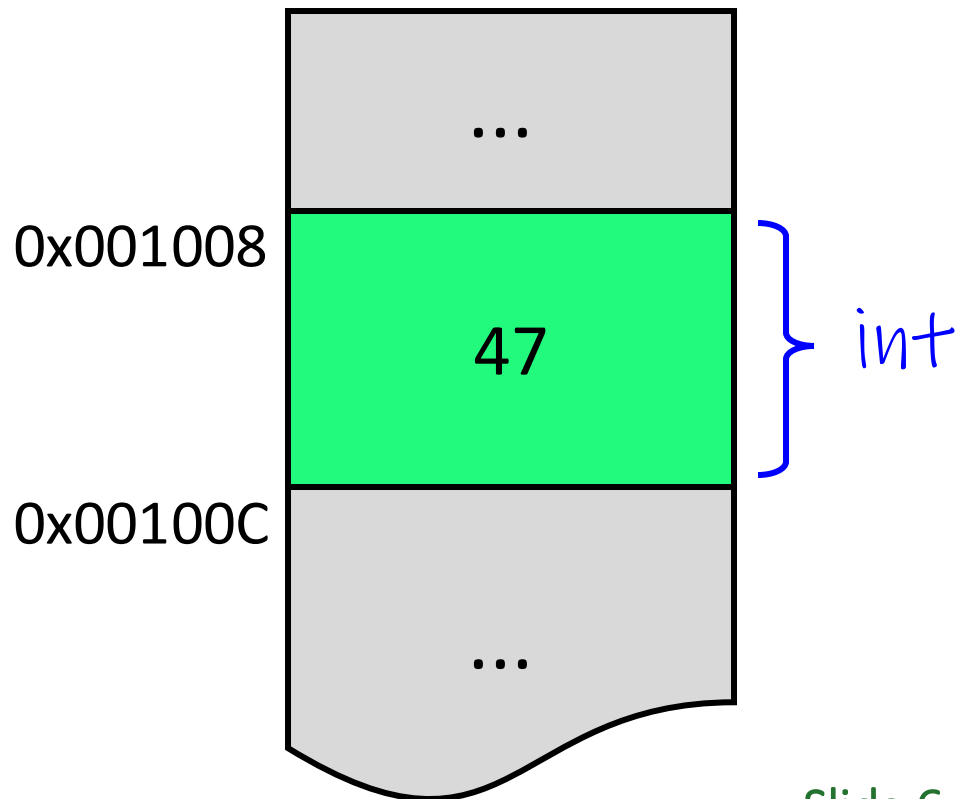
Variáveis e ponteiros

- A forma mais básica para passar um endereço de memória para um ponteiro é usando o operador de endereço (&) em um variável e atribuindo ao ponteiro.

```
int *p;
```

```
int x = 47;
```

```
p = &x;
```



Variáveis e ponteiros

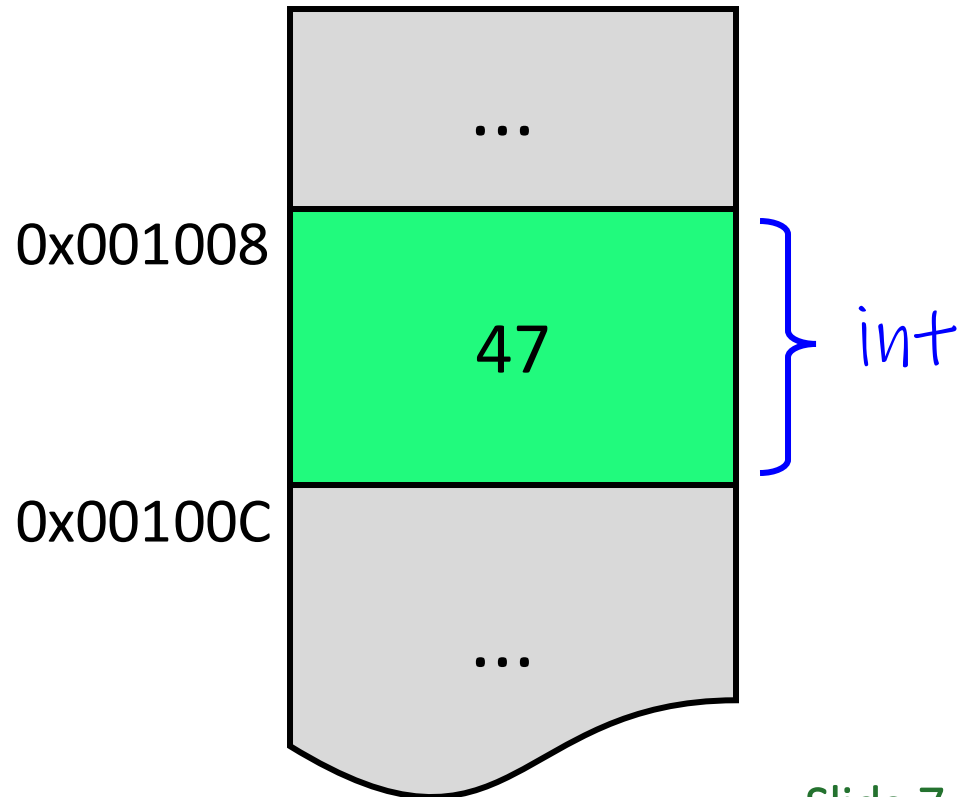
- É possível acessar o conteúdo presente no endereço apontado pelo ponteiro, usando o operador asterisco (*) após a declaração do ponteiro.

```
int *p;
```

```
int x = 47;
```

```
p = &x;
```

```
cout << *p;
```



Variáveis e ponteiros

- Existem duas formas para acessar o conteúdo apontado por um ponteiro para tipos estruturados (classes e structs): o jeito fácil e o jeito difícil...

Variáveis e ponteiros

Jeito difícil

```
class Circulo
{
public:

    int a;

    Circulo();
    GetArea();
    ...
};
```

```
...
Circulo circ;

Circulo *x;

x = &circ;

(*x).a = 47;

cout << (*x).GetArea();
...
```

Variáveis e ponteiros

Jeito fácil

```
class Circulo
{
public:

    int a;

    Circulo();
    GetArea();
    ...
};
```

```
...
Circulo circ;

Circulo *x;

x = &circ;

x->a = 47;

cout << x->GetArea();
...
```

Ponteiros

Aritmética de ponteiros

Aritmética de ponteiros

- A **aritmética de ponteiros** envolve operações como adição, subtração, incremento e decremento diretamente sobre valores de endereços de memória.

Aritmética de ponteiros

Incremento de ponteiros

- Quando você incrementa um ponteiro, ele aponta para o próximo elemento do tipo ao qual ele está apontando.

```
int *p;
```

```
int x = 47;
```

```
p = &x;
```

```
p++;
```

0x001008

0x001009

0x00100A

0x00100B

0x00100C

...

47

...

} int

Aritmética de ponteiros

Decremento de ponteiros

- Quando você decrementa um ponteiro, ele aponta para o elemento anterior do tipo ao qual ele está apontando.

```
int *p;
```

```
int x = 47;
```

```
p = &x;
```

```
p--;
```

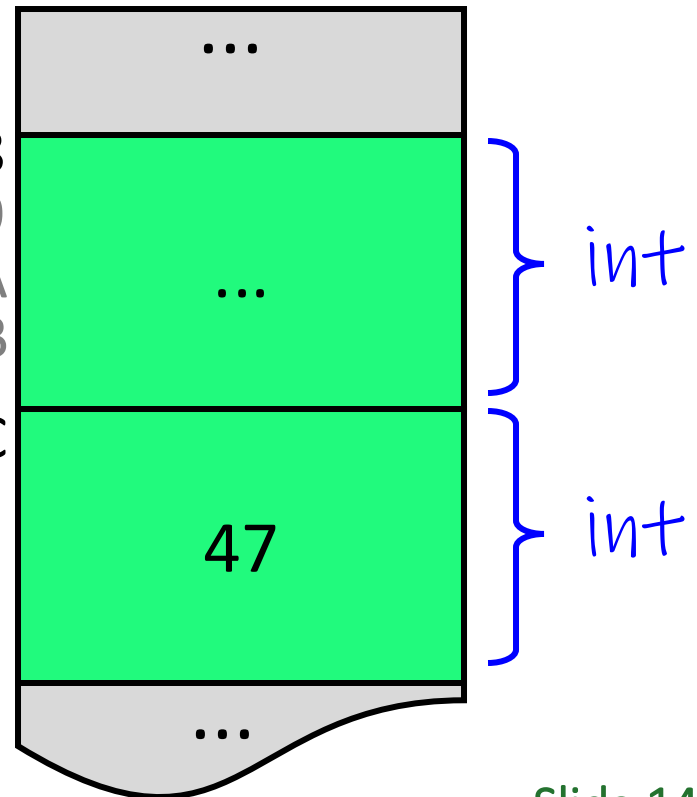
0x001008

0x001009

0x00100A

0x00100B

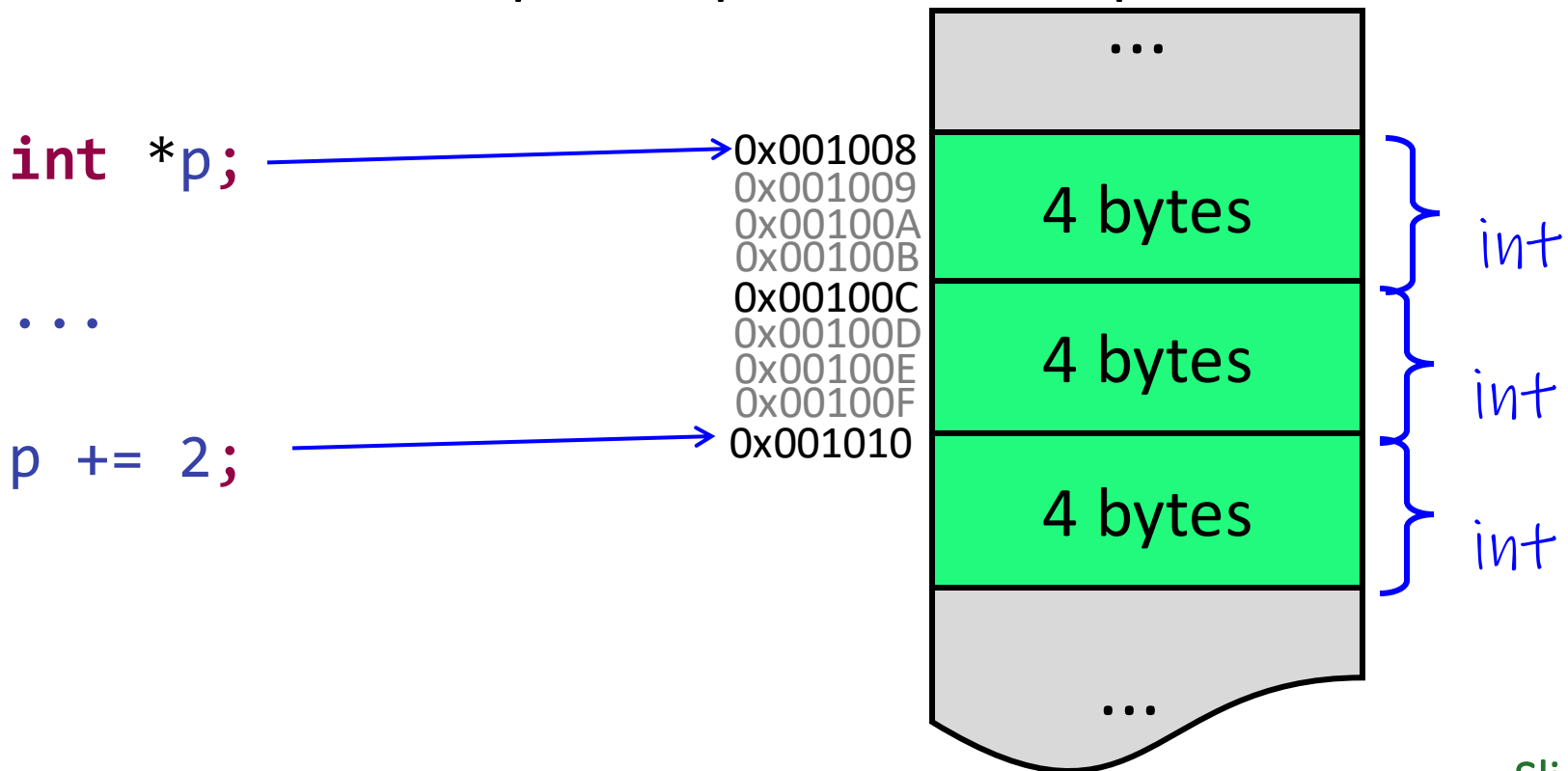
0x00100C



Aritmética de ponteiros

Adição em ponteiros

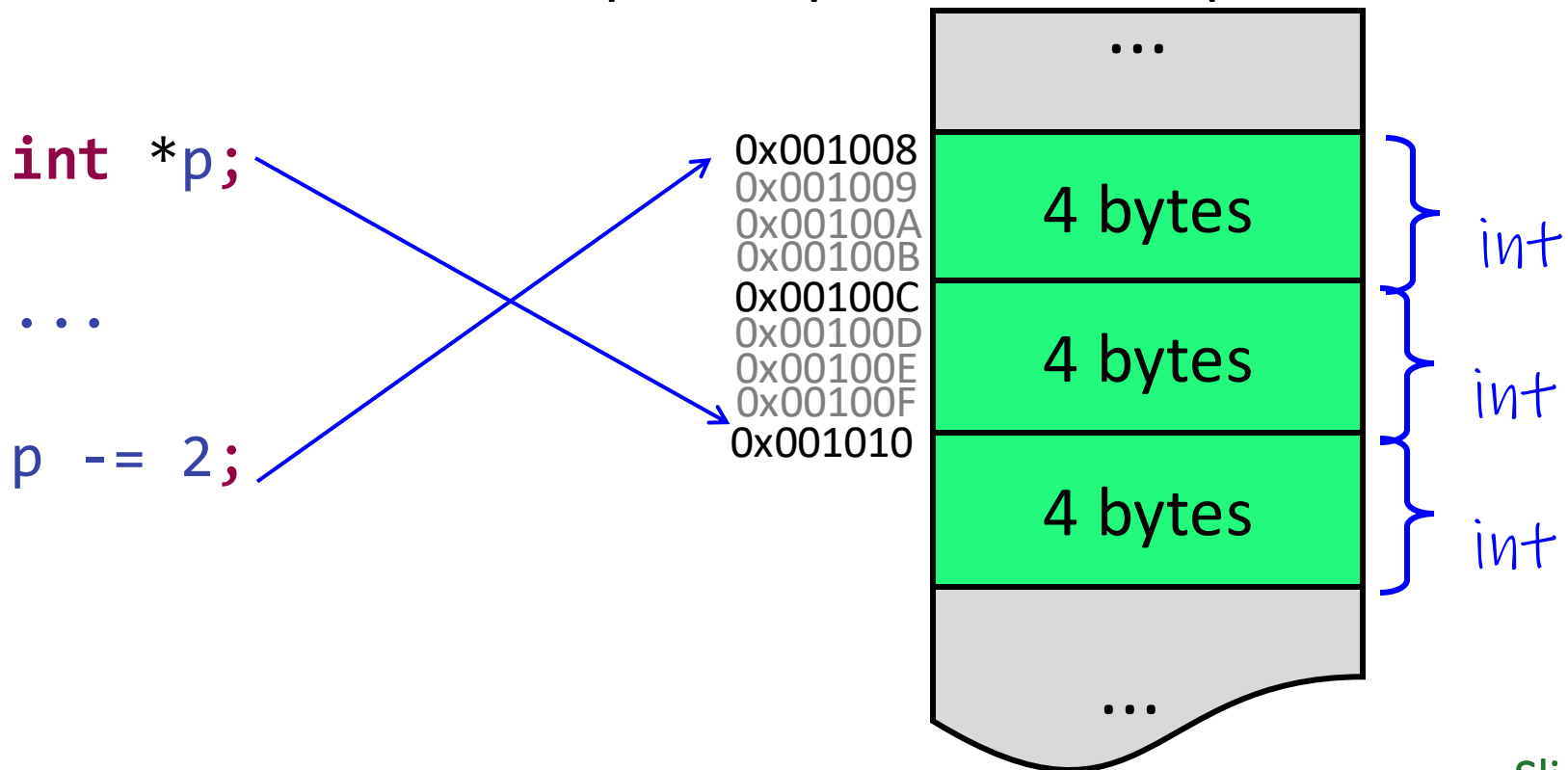
- Você pode adicionar um número inteiro a um ponteiro para avançar o ponteiro por múltiplos do tamanho do tipo ao qual ele está apontando.

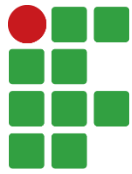


Aritmética de ponteiros

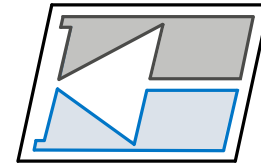
Subtração em ponteiros

- Você pode subtrair um número inteiro de um ponteiro para retroceder o ponteiro por múltiplos do tamanho do tipo ao qual ele está apontando.





INSTITUTO FEDERAL
Santa Catarina



DAELN

Ponteiros

Vetores e ponteiros

Vetores e ponteiros

- Em C++, vetores (arrays) e ponteiros estão intimamente relacionados.
- O nome de um vetor pode ser utilizado como um ponteiro constante para o primeiro elemento do vetor.

Exemplo:

```
int vet[5] = {7, 66, 39, 45, 0};
```

```
int *p = vet; // vet == &vet[0]
```

Vetores e ponteiros

- Podemos acessar cada elemento do vetor por meio de um ponteiro para o seu primeiro elemento.

Exemplo:

```
int vet[5] = {7, 66, 39, 45, 0};
```

```
int *p = vet; // vet == &vet[0]
```

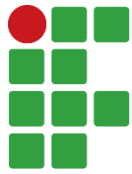
```
cout << *p; // Imprime vet[0] = 7
```

```
cout << *(p + 1); // Imprime vet[1] = 66
```

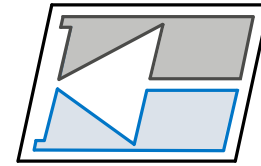
```
cout << *(p + 2); // Imprime vet[2] = 39
```

```
cout << *(p + 3); // Imprime vet[3] = 45
```

```
cout << *(p + 4); // Imprime vet[4] = 0
```



INSTITUTO FEDERAL
Santa Catarina



DAELN

Ponteiros

O ponteiro do tipo "void *"

O ponteiro do tipo "void *"

- O ponteiro do tipo **void *** é um tipo especial de ponteiro em C++ que pode apontar para qualquer tipo de dado.
- Características:
 - **Generalidade:** pode armazenar o endereço de qualquer tipo de dado;
 - **Conversão explícita:** para acessar o conteúdo do ponteiro ou usar aritmética de ponteiros, é necessário converter para o tipo desejado;
 - **Utilização comum em funções de biblioteca:** amplamente utilizados em funções de biblioteca padrão.

O ponteiro do tipo "void *"

```
#include <iostream>
using namespace std;

enum setaValorTipo
{
    TIPO_EH_CHAR = 0,
    TIPO_EH_INT,
    TIPO_EH_LONG_LONG,
    TIPO_EH_FLOAT,
    TIPO_EH_DOUBLE
};
```

```
int main()
{
    int x1, x2 = 3;
    char c1, c2 = 'A';

    seta_valor( &x2, &x1, TIPO_EH_INT);
    seta_valor( &c2, &c1, TIPO_EH_CHAR);

    cout << "x1 = " << x1 << endl;
    cout << "c1 = " << c1 << endl;
}
```

```
void seta_valor( void *origem, void *dest, setaValorTipo tipo)
{
    switch (tipo)
    {
        case TIPO_EH_CHAR:
            char *origem_tmp = reinterpret_cast<char*>(origem);
            char *dest_tmp = reinterpret_cast<char*>(dest);
            *dest_tmp = *origem_tmp;
            break;
        case TIPO_EH_INT:
            int *origem_tmp = reinterpret_cast<int*>(origem);
            int *dest_tmp = reinterpret_cast<int*>(dest);
            *dest_tmp = *origem_tmp;
            break;
        ...
        default:
            break;
    }
}
```

Uso do void* em parâmetro de funções

Ponteiros

Ponteiro para funções

Ponteiro para funções

- A memória do computador é separada logicamente em **região de dados** e **região de programa**.
- Os ponteiros vistos até agora apontam para dados, ou seja, para a região de dados da memória.
- Contudo, podemos ter um ponteiro que aponta para uma função, ou seja, para uma região de programa.

Ponteiro para funções

- Para declarar um ponteiro para uma função, você deve especificar o tipo de retorno da função e a lista de parâmetros que a função aceita.
- A sintaxe básica é:

```
retorno (*nome_do_ponteiro)(tipo1, tipo2, ...);
```

Ponteiro para funções

Exemplo:

```
int soma(int a, int b) {  
    return a + b;  
}
```

Declaração da função

```
int (*ptrFuncao)(int, int);
```

Declaração de um
ponteiro para uma
função com protótipo
igual á função acima

```
ptrFuncao = soma;
```

Fazendo com que o
ponteiro aponte para
função específica.

Ponteiro para funções

Chamada de funções usando ponteiros

- Uma vez que você tenha atribuído uma função ao ponteiro, pode chamar a função usando o ponteiro da mesma forma que chamaria a função diretamente:

```
int soma(int a, int b) {  
    return a + b;  
}
```

```
int (*ptrFuncao)(int, int);
```

```
ptrFuncao = soma;
```

```
int resultado = ptrFuncao(3, 4); // Chama a função soma(3, 4)
```

```
#include <iostream>
```

```
using namespace std;
```

```
int soma(int a, int b) {  
    return a + b;  
}
```

```
int subtracao(int a, int b) {  
    return a - b;  
}
```

```
void executarOperacao(int (*operacao)(int, int), int x, int y) {  
    cout << "Resultado: " << operacao(x, y) << endl;  
}
```

```
int main() {  
    executarOperacao(soma, 10, 5);           // Chama soma(10, 5)  
    executarOperacao(subtracao, 10, 5);      // Chama subtracao(10, 5)  
    return 0;  
}
```

- Você pode passar ponteiros para funções como parâmetros para outras funções.